

Machine Learning Libraries — Complete Beginner Guide

Author: Yusufvali

Important Python libraries used in Machine Learning, Data Science, Deep Learning, NLP, and Computer Vision — with simple examples.

Library	Main Purpose	Priority
NumPy	Numerical calculations, arrays, matrices	★★★★★
Pandas	Data loading, cleaning, analysis	★★★★★
Matplotlib	Basic data visualization	★★★★■
Seaborn	Statistical visualization	★★★★■
Scikit-learn	Traditional Machine Learning	★★★★★
SciPy	Scientific computing and optimization	★★★██
Statsmodels	Statistics and statistical models	★★★██
TensorFlow	Deep Learning	★★★★■
PyTorch	Deep Learning and AI	★★★★★
Keras	Easy neural-network API	★★★★■
Transformers	Pretrained NLP/LLM models	★★★★★
OpenCV	Computer Vision and image processing	★★★★■

1. NumPy

Numerical calculations, arrays, matrices and mathematical operations.

```
import numpy as np

numbers = np.array([10, 20, 30, 40])
print(np.mean(numbers))
print(np.sum(numbers))
```

2. Pandas

Working with structured data such as CSV and Excel files.

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head())
print(df["salary"].mean())
```

3. Matplotlib

Creating basic charts and visualizing relationships.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

plt.plot(x, y)
plt.xlabel("x")
```

```
plt.ylabel("Y")
plt.show()
```

4. Seaborn

Statistical visualization such as heatmaps, distributions and box plots.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()
```

5. Scikit-learn

Core traditional ML library: preprocessing, algorithms, model selection and metrics.

```
from sklearn.linear_model import LinearRegression

X = [[1], [2], [3], [4]]
y = [2, 4, 6, 8]

model = LinearRegression()
model.fit(X, y)

prediction = model.predict([[5]])
print(prediction)
```

6. SciPy

Scientific computing, optimization, statistics and linear algebra.

```
from scipy import stats

x = [10, 20, 30, 40]
print(stats.describe(x))
```

7. Statsmodels

Detailed statistical modeling, regression summaries, p-values and confidence intervals.

```
import statsmodels.api as sm

X = [1, 2, 3, 4]
y = [2, 4, 6, 8]

X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())
```

8. TensorFlow

Deep learning framework for neural networks and large-scale AI applications.

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation="relu"),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer="adam", loss="mse")
```

9. PyTorch

Deep learning framework widely used in AI research, computer vision and NLP.

```
import torch
import torch.nn as nn

model = nn.Linear(1, 1)

x = torch.tensor([[2.0]])
y = model(x)
print(y)
```

10. Keras

High-level API that makes neural-network development easier.

```
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1)
])

model.summary()
```

11. Hugging Face Transformers

Pretrained transformer models for NLP, text classification, summarization and LLM applications.

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
print(classifier("I love machine learning!"))
```

12. OpenCV

Computer vision, image processing, video processing and image transformations.

```
import cv2

image = cv2.imread("photo.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

cv2.imwrite("gray.jpg", gray)
```

Recommended ML Learning Order

Step 1: Python basics → **Step 2:** NumPy → **Step 3:** Pandas → **Step 4:** Matplotlib/Seaborn → **Step 5:** Scikit-learn → **Step 6:** Regression & Classification → **Step 7:** Model Evaluation → **Step 8:** PyTorch/TensorFlow → **Step 9:** Transformers/GenAI → **Step 10:** Projects.

Quick Memory

NumPy = numbers & arrays

Pandas = data

Matplotlib/Seaborn = graphs

Scikit-learn = ML

PyTorch/TensorFlow = Deep Learning

Transformers = NLP/LLMs

OpenCV = Computer Vision

Author: Yusufvali